

# Fiche : agents, tokens et familles de modèles IA

Oct 7, 2026 · @gab

## 1. OpenClaw et les « super-agents »

Un agent IA qui peut tout faire devient puissant, mais aussi ingérable et dangereux : c'est l'idée centrale de cette partie.

- **OpenClaw** : agent IA open source capable d'agir sur l'ordinateur à la place de l'utilisateur (fichiers, messageries, navigateur, outils). Son créateur en a fait un phénomène à lui seul.
- **À vérifier** : les notes parlent d'un rachat par OpenAI à 1 Md\$. À ma connaissance, OpenAI a plutôt recruté son créateur, et le projet continue en open source sous une fondation.
- **Agent** : un LLM en boucle (observer, raisonner, agir avec des outils, recommencer) jusqu'à ce que la tâche soit terminée.
- **Super-agent** : plus il a de permissions, plus les risques augmentent. Erreurs en cascade, actions irréversibles, fuites de données et *prompt injection* (un texte malveillant lu par l'agent lui fait exécuter des ordres) deviennent difficiles à maîtriser.

## 2. « Les interfaces vont disparaître, et les logiciels aussi »

La thèse : l'humain n'ira plus cliquer dans des logiciels, il exprimera une intention et un agent fera le travail.

- **Aujourd'hui** : on utilise des interfaces (boutons, menus, écrans).
- **Demain** : on formule une demande en langage naturel, et l'agent appelle directement les services via des **API** ou des protocoles comme **MCP**.
- **MCP** (*Model Context Protocol*) : standard ouvert, lancé par Anthropic fin 2024, qui permet à une application d'IA de se connecter de façon uniforme à des outils et des sources de données externes (fichiers, bases de données, Slack, Gmail), un peu comme un « USB-C pour l'IA ».
- **Conséquence** : la valeur se déplace de l'interface (le SaaS) vers les **données**, les **règles métier** et l'**exécution**.
- **Nuance** : c'est une thèse, pas une certitude. Les logiciels risquent surtout de devenir invisibles, des « back-ends » pour agents.

### 3. Les tokens : où vont-ils ?

Un agent consomme des milliards de tokens parce qu'il lit, cherche et écrit en boucle.

- **Token** : unité de texte traitée par le modèle, un morceau de mot. En anglais, 1 000 tokens valent environ 750 mots. Les API facturent au token, en entrée et en sortie.
- **Exemple des notes** : V. Bernard a consommé 13 milliards de tokens en un mois (septembre). Les agents relisent leur contexte à chaque étape, d'où ces volumes.
- **Trois postes de consommation** :
  1. **Comprendre / lire** : la consigne et le contexte.
  2. **Scraper et chercher** : aller chercher de l'information (web, documents). C'est souvent le plus gros poste, car les pages lues sont longues.
  3. **Produire l'output** : la réponse, souvent plus chère au token que l'entrée.
- **Chinois contre langues occidentales** : un caractère chinois porte plus de sens qu'une lettre latine, donc la même idée s'exprime avec moins de symboles. Nuance : tout dépend du **tokenizer** (le découpeur de texte). Le français consomme en général plus de tokens que l'anglais pour un même contenu.

### 4. La fenêtre de contexte (*context window*)

La fenêtre de contexte est la mémoire de travail du modèle, et elle a des limites qu'il faut gérer par des résumés.

- **Définition** : tout ce que le modèle peut « avoir sous les yeux » dans une session (conversation, documents, résultats d'outils). Hors de cette fenêtre, il ne sait rien.
- **Taille** : les modèles frontières atteignent environ **1 M de tokens**, soit une dizaine de romans.
- **Limite** : ils n'exploitent pas tout aussi bien. Les infos du **milieu** sont moins bien retrouvées (phénomène *lost in the middle*) et la qualité baisse quand le contexte est très chargé (*context rot*). Les modèles se souviennent de certains passages, pas de tous.
- **Parade : la compaction** (*compacting conversation*). On demande au modèle de résumer la conversation, puis on repart d'un contexte propre contenant seulement ce résumé. Cela réduit les coûts et la dérive.

### 5. Les familles de modèles

Il existe au moins trois grandes approches, chacune adaptée à un type de problème.

#### ① LLM (*Large Language Models*)

- Ils prédisent le **token suivant** d'après des statistiques apprises sur d'énormes corpus de textes.
- Excellents pour les tâches intellectuelles : écrire, résumer, coder, raisonner sur du texte.

## ② World Models (thèse de Yann LeCun)

- Ils apprennent une **représentation du monde** (physique, causalité, évolution des situations) plutôt que des corrélations de texte. Ils sont entraînés notamment sur la vidéo et les capteurs, et simulent « ce qui se passe si... ».
- LeCun défend que les LLM sont limités, car ils ne comprennent pas vraiment le monde physique. Il faut donc des world models, utiles pour la **robotique** et les **voitures autonomes**.
- Son architecture de référence s'appelle **JEPA** (*Joint Embedding Predictive Architecture*). Elle prédit dans un espace abstrait plutôt que pixel par pixel.

## ③ « JEV » : modèle « Système 1 »

- Les notes décrivent un modèle de type **arbre de décision**, oui ou non, sans erreurs et très économe en tokens. Elles font référence au « Système 1 » de Kahneman : pensée rapide et automatique, par opposition au Système 2, lent et réfléchi.

# 6. Les limites des LLM

Les LLM sont chers et ne peuvent pas garantir l'exactitude de leurs réponses, ce qui bloque certains usages critiques.

- **Très coûteux** : l'entraînement et l'inférence demandent beaucoup de GPU, d'énergie et de tokens.
- **Hallucinations** : le modèle génère du plausible, pas du vrai. On **ne peut pas prédire à 100 %** ses réponses, ce qui est un problème en finance, en santé ou en défense.
- **Parades** : RAG (retrouver des sources fiables avant de répondre), appel d'outils, vérification humaine, modèles plus petits et spécialisés.
- **Exemple noté** : « Hugging Face (escaping) » reste ambigu. Hugging Face est la plateforme de référence des modèles open source, ce qui peut illustrer l'échappatoire aux coûts avec des modèles ouverts et plus légers. À confirmer.

# 7. Glossaire : concepts techniques clés

Neuf notions reviennent partout : RAG, embedding, inférence, température, open weights, CLI, terminal, orchestration et harness.

## RAG (*Retrieval-Augmented Generation*)

- **Définition** : technique qui donne au LLM des **documents pertinents au moment de la question**, au lieu de compter uniquement sur ce qu'il a appris à l'entraînement. En français : « génération augmentée par la recherche ».
- **Fonctionnement** :
  1. La question est transformée en **embedding**.
  2. On cherche dans une **base vectorielle** les passages dont le sens est le plus proche.
  3. Ces passages sont **insérés dans le prompt**, dans la fenêtre de contexte.
  4. Le LLM répond en s'appuyant sur ces sources.
- **Intérêt** : réduit les hallucinations, donne accès à des données récentes ou privées **sans réentraîner** le modèle, et permet de citer ses sources.
- **Limites** : si la recherche ramène de mauvais passages, la réponse sera mauvaise. Les passages injectés consomment aussi des tokens, donc du coût.

## Embedding (« plongement »)

- **Définition** : représentation d'un texte (ou d'une image, d'un son) sous forme d'une **liste de nombres**, un vecteur, qui capture son **sens**. Les dimensions vont de quelques centaines à plusieurs milliers.
- **Principe** : deux contenus de sens proche ont des vecteurs **proches**. « Chien » et « chat » sont voisins, « chien » et « obligation convertible » sont éloignés. La proximité se mesure par la **similarité cosinus**.
- **Usages** : recherche sémantique (par le sens plutôt que par mots-clés), recommandations, classification et surtout RAG.
- **Stockage** : dans une **base vectorielle** (FAISS, Pinecone, etc.).
- **Lien avec LeCun** : JEPA prédit dans un « espace abstrait », c'est-à-dire un espace d'embeddings, plutôt que pixel par pixel.

## Inférence (*inference*)

- **Définition** : phase où un modèle **déjà entraîné** est **utilisé** pour produire une réponse. Chaque prompt envoyé déclenche de l'inférence.
- **Entraînement** : on ajuste les poids du modèle sur d'énormes données. C'est très coûteux, ponctuel, et réalisé par les labs.
- **Inférence** : on exécute le modèle figé. Le coût par requête est faible, mais il se répète des millions de fois.
- **Lien avec les notes** : c'est ce qui est **facturé au token**. Les agents et les modèles de raisonnement consomment beaucoup d'inférence, ce qui explique les 13 Md de tokens en un mois et le caractère très coûteux des LLM.

- **Enjeux** : latence, coût par token, GPU. L'optimisation de l'inférence est devenue un marché à part entière.

## Température

- **Définition** : paramètre qui règle le **degré de hasard** dans le choix du token suivant au moment de l'inférence.
- **Basse (0 à 0,3)** : réponses stables et factuelles. **Haute (0,8 et plus)** : réponses variées et créatives, mais plus risquées.
- **Techniquement** : on divise les scores bruts (*logits*) par la température avant le *softmax*. Une température basse « pique » la distribution, une haute l'« aplatit ».
- **Attention** : elle règle la **régularité**, pas la **vérité**. À 0, un modèle peut répéter la même erreur à l'identique, donc elle ne supprime pas les hallucinations.
- **Paramètres voisins** : *top-p* (on tire parmi les tokens couvrant p % de probabilité cumulée) et *top-k* (on ne garde que les k tokens les plus probables).

## Open weights (« poids ouverts »)

- **Définition** : un modèle dont les **poids** (les paramètres appris pendant l'entraînement) sont **publiés et téléchargeables**. N'importe qui peut le faire tourner sur ses propres machines, l'adapter ou le déployer.
- **À ne pas confondre avec l'open source** : en général, seuls les poids sont publiés. Le code d'entraînement, les données et la recette complète restent souvent fermés, et la **licence** peut limiter certains usages (usage commercial, taille de l'entreprise). Un modèle open source au sens strict publie tout.
- **Opposé** : les modèles **fermés** (propriétaires), accessibles uniquement via une API payante au token. On ne voit jamais leurs poids.
- **Exemples** : Llama (Meta), Mistral, Qwen et DeepSeek (Chine), gpt-oss (OpenAI). Ils sont distribués sur **Hugging Face**, ce qui rejoint ta note sur la plateforme.
- **Intérêt** : **confidentialité** (les données ne quittent pas l'entreprise, crucial en finance ou en défense), pas de facturation au token (mais il faut payer les GPU), possibilité de **fine-tuning** (réentraîner légèrement le modèle sur ses données) et indépendance vis-à-vis d'un fournisseur.
- **Limites** : il faut l'infrastructure et les compétences pour les héberger, ils sont souvent légèrement en retrait des meilleurs modèles fermés, et leurs garde-fous de sécurité peuvent être retirés par n'importe qui.
- **Lien avec les notes** : c'est une réponse au caractère **très coûteux** des LLM et à la dépendance aux grands labs. Les modèles chinois ouverts, très optimisés en tokens, y jouent un rôle central.

## Terminal

- **Définition** : la fenêtre où l'on pilote un ordinateur en **tapant des commandes** au lieu de cliquer. Sous Windows : PowerShell ou l'invite de commandes. Sous Mac et Linux : Terminal, avec un *shell* (bash, zsh).
- **Intérêt pour l'IA** : un agent qui a accès au terminal peut tout faire comme un développeur : créer des fichiers, lancer du code, installer des outils, interroger des API. C'est ce qui le rend puissant, et donc risqué (voir le point 1 sur les super-agents).

## CLI (*Command Line Interface*)

- **Définition** : un programme qu'on utilise **en ligne de commande**, depuis le terminal, plutôt que par une interface graphique. Exemple : `git commit` ou `claude` pour lancer Claude Code.
- **Terminal ou CLI ?** Le **terminal** est la fenêtre, la **CLI** est le type d'outil qu'on y lance.
- **Pourquoi les agents l'aiment** : une CLI est du texte en entrée et du texte en sortie, ce qui convient parfaitement à un LLM. Elle est aussi facile à enchaîner et à automatiser, ce qui rejoint la thèse « les interfaces vont disparaître ».
- **Exemples d'agents en CLI** : Claude Code, Codex CLI, Gemini CLI.

## Orchestration

- **Définition** : la **coordination** de plusieurs étapes, outils, modèles ou agents pour accomplir une tâche complexe. Elle décide qui fait quoi, dans quel ordre, avec quelles données, et gère les erreurs et les nouvelles tentatives.
- **Exemple** : un agent « chef d'orchestre » découpe une demande en sous-tâches, confie chacune à un **sous-agent** (recherche, rédaction, vérification), puis assemble les résultats. Il peut aussi **router** : un petit modèle bon marché pour les tâches simples, un grand modèle pour les tâches difficiles.
- **Intérêt** : elle permet d'économiser des tokens, de paralléliser le travail et de garder chaque contexte plus court et plus propre (voir la compaction au point 4).
- **Outils** : des frameworks comme LangChain ou LangGraph, ou une logique écrite à la main.
- **Limite** : plus il y a d'agents, plus les erreurs se propagent et plus la facture de tokens grimpe.

## Harness (« harnais »)

- **Définition** : tout ce qui est construit **autour du modèle** pour en faire un agent utilisable. Le modèle seul ne fait que prédire du texte, c'est le harness qui lui donne les moyens d'agir.

- **Ce qu'il contient** : la **boucle** de l'agent (observer, raisonner, agir), les **outils** (fichiers, web, terminal), les **permissions**, la gestion du contexte (compaction, mémoire), les instructions système et la vérification des résultats.
- **Formule utile** : *agent = modèle + harness*. Claude Code, Codex CLI ou OpenClaw sont des harness.
- **Pourquoi c'est important** : à modèle égal, un meilleur harness donne de bien meilleurs résultats. C'est aussi lui qui pose les garde-fous (demander une validation avant une action sensible).
- **Harness et orchestration** : le harness est l'environnement complet d'un agent, l'orchestration est la coordination de plusieurs agents ou étapes, souvent construite à l'intérieur d'un harness.

## Comment ces notions s'enchaînent

1. La question devient un **embedding**.
2. Une recherche dans la base ramène les passages pertinents (**RAG**).
3. Ils sont ajoutés au contexte.
4. Le LLM génère la réponse (**inférence**, avec une certaine **température**).
5. Le coût se compte en **tokens**.